

Nüfus ve Sosyal Araştırmalar Enstitüsü

Yapay Zeka Modülü | Makine Öğrenmesi — 8. Ders

Naïve Bayes & Random Forest

Ders: Sebnem Paluluoglu — Veri Analizi Okulu, Bogazici Universitesi

Ekstra Referans: Yahya et al. (2025) *Bias Detection in Media: Traditional Models vs. Transformers* (ACL Workshop)

Bu ders notunun amacı:

Ders slaytında islenen teorik konuları adım adım Python koduyla görmek, formüller arkasındaki sezgiyi kavramak ve scikit-learn ile pratik yapmaktır.

Ekstra: etiketi ders kapsamı dışında fakat önemli akademik bilgileri işaretler.

0. Kutuphane Yüklemesi

Tüm kutuphaneler standart scikit-learn ekosistemindedir.

Eksik varsa ilk satiri aktif edin.

```
In [1]: # Bu hücre internet erişimi gerektirir.
# Aşağıdaki kodu kendi ortamında internet varken çalıştırın.
# Notebook bu hücre olmadan da tam çalışır.
# from sklearn.datasets import fetch_20newsgroups
# # İlk çalıştırmada aşağıdaki satiri aktif edin:
!pip install scikit-learn pandas numpy matplotlib seaborn --quiet

import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.patches import Circle, Patch

from sklearn.datasets import load_wine, load_breast_cancer, fetch_20newsgroups
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12
sns.set_style('whitegrid')
print('Tum kutuphaneler basariyla yuklendi!')
```

Tum kutuphaneler basariyla yuklendi!

BOLUM 1: NAIVE BAYES

Naive Bayes, **olasilik teorisine** dayali bir siniflandirma algoritmasıdır.

Ozellikle **metin siniflandirma** gorevlerinde guclu performans gosterir:

- E-posta spam tespiti
- Haber kategorilendirme (spor, ekonomi, politika...)
- Duygu analizi (yorum pozitif mi, negatif mi?)

Algoritmanın temeli iki adimdan olusur:

1. **Kosullu Olasilik** — bir bilgi verilmisken baska bir sey in olasiligini hesapla
2. **Bayes Teoremi** — bu olasiklari birlestirerek sinif tahmini yap

1.1 Kosullu Olasilik (Conditional Probability)

Kosullu olasilik, **B zaten gerceklesmisse A'nin gerceklesme olasiligi** demektir.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Formulu okuma sekli: "B biliniyorken A'nin olasiligi
= ikisinin birden gerceklesme olasiligi bolü B'nin olasiligi"

Slayttaki Ornek (12.000 katilimci):

- A = Temel Istatistik Modulune Katilanlar = 8.000 kisi
- B = Yapay Zeka Modulune Katilanlar = 9.000 kisi
- A kesim B = Her ikisine birden katilanlar = 5.000 kisi

$$P(A|B) = \frac{5000}{9000} \approx 0.556$$

Yorum: YZ modülüne katılmış birinin Temel İstatistik'e de katılmış olma olasılığı **%55.6**

Önemli nokta: Bu bilgi olmadan tahminimiz $8000/12000 = \%66.7$ 'ydi.

B bilgisi eklenince tahmin değişti — Bayes'in ozu budur!

```
In [2]: # — Kosullu Olasılık: Venn Diyagramı —————

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))

# Sol panel: Venn Diyagramı
ax1.set_xlim(0, 10)
ax1.set_ylim(0, 8)
ax1.set_aspect('equal')

daire_A = Circle((3.5, 4), 2.5, color='#AED6F1', alpha=0.65)
daire_B = Circle((6.5, 4), 2.5, color='#A9DFBF', alpha=0.65)
ax1.add_patch(daire_A)
ax1.add_patch(daire_B)

ax1.text(2.2, 4, '3.000', ha='center', va='center', fontsize=13, fontweight='bold',
ax1.text(5.0, 4, '5.000', ha='center', va='center', fontsize=13, fontweight='bold',
ax1.text(7.8, 4, '4.000', ha='center', va='center', fontsize=13, fontweight='bold',
ax1.text(2.0, 6.8, 'A: Temel\nİstatistik', ha='center', fontsize=10, color='#1A5276',
ax1.text(8.0, 6.8, 'B: Yapay\nZeka', ha='center', fontsize=10, color='#1E8449', fon
ax1.text(5.0, 2.5, 'A ve B\n(Her ikisi)', ha='center', fontsize=9, color='#7D6608')
ax1.set_title('Kosullu Olasılık: Venn Diyagramı', fontsize=13, fontweight='bold')
ax1.axis('off')

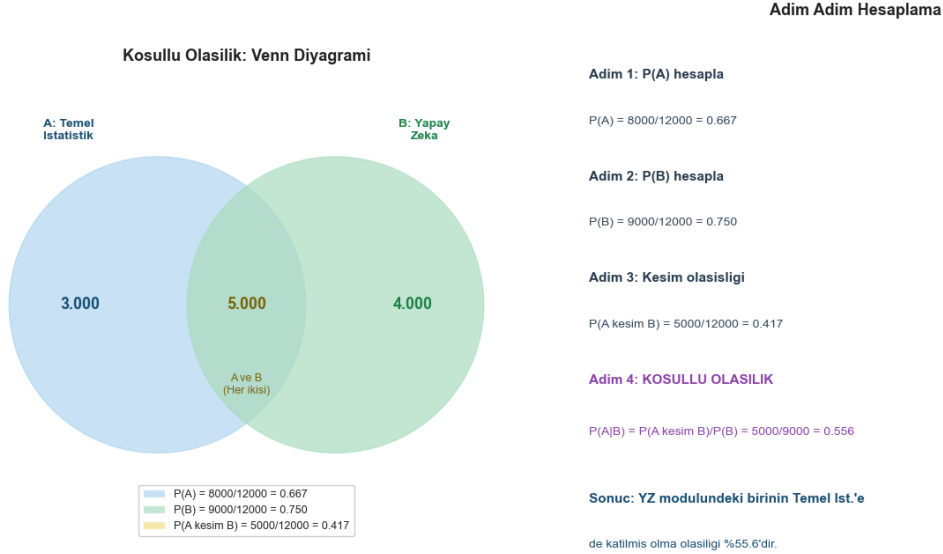
legend_elems = [
    Patch(facecolor='#AED6F1', alpha=0.65, label='P(A) = 8000/12000 = 0.667'),
    Patch(facecolor='#A9DFBF', alpha=0.65, label='P(B) = 9000/12000 = 0.750'),
    Patch(facecolor='#F9E79F', alpha=0.85, label='P(A kesim B) = 5000/12000 = 0.417')
]
ax1.legend(handles=legend_elems, loc='lower center', fontsize=9, framealpha=0.9)

# Sağ panel: Adım adım hesaplama
ax2.axis('off')
ax2.set_title('Adım Adım Hesaplama', fontsize=13, fontweight='bold')

adimlar = [
    ("Adım 1: P(A) hesapla", 0.90, '#2C3E50'),
    ("P(A) = 8000/12000 = 0.667", 0.82, '#34495E'),
    ("Adım 2: P(B) hesapla", 0.72, '#2C3E50'),
    ("P(B) = 9000/12000 = 0.750", 0.64, '#34495E'),
    ("Adım 3: Kesim olasılığı", 0.54, '#2C3E50'),
    ("P(A kesim B) = 5000/12000 = 0.417", 0.46, '#34495E'),
    ("Adım 4: KOSULLU OLASILIK", 0.36, '#8E44AD'),
    ("P(A|B) = P(A kesim B)/P(B) = 5000/9000 = 0.556", 0.27, '#8E44AD'),
    ("Sonuç: YZ modulundeki birinin Temel İst.'e", 0.15, '#1A5276'),
    ("de katılmış olma olasılığı %55.6'dır.", 0.07, '#1A5276'),
]
```

```
for metin, y, renk in adimlar:
    fontsize = 11 if 'Adim' in metin or 'Sonuc' in metin else 10
    fw = 'bold' if 'Adim' in metin or 'Sonuc' in metin else 'normal'
    ax2.text(0.05, y, metin, transform=ax2.transAxes, fontsize=fontsize,
            color=renk, fontweight=fw)

plt.tight_layout()
plt.savefig('kosullu_olasilik.png', dpi=120, bbox_inches='tight')
plt.show()
print("Gorsel kaydedildi: kosullu_olasilik.png")
```



Gorsel kaydedildi: kosullu_olasilik.png

1.2 Bayes Teoremi

İki kosullu olasilik formulu birlestirince **Bayes Teoremi** ortaya cilar:

$$P(A|B) \cdot P(B) = P(A \cap B) = P(B|A) \cdot P(A)$$

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Teoremi olusturan dort bilezen ve makine ogrenmesindeki karsiliklari:

| Terim | Isim | Makine Ogrenmesindeki Anlam | |-----|-----|-----| |
 $P(A|B)$ | **Posterior** | Veriyi gordukten sonra guncellenen sinif olasiligi | | $P(B|A)$ |
Likelihood | Bu sinif bu kelimeleri uretir miydi? | | $P(A)$ | **Prior** | Egitim setinde bu sinif ne
kadar yaygin? | | $P(B)$ | **Marginalization** | Normallestime sabiti — karsilastirmada iptal olur |

Sezgi ile ornek: Bir doktora gidiyorsunuz, atesiniz var.

- Prior: Grip bu mevsimde ne kadar yaygin? $P(\text{grip})$
- Likelihood: Grip olanlarda ates ne siklikla gorulur? $P(\text{ates}|\text{grip})$

- Posterior: Atesi oldugunu bildigimizde grip ihtimali? $P(\text{grip}|\text{ates})$

Bayes teoremi "yeni kaniti ($B=\text{ates}$) eski inanca ($P(\text{grip})$) ekleyerek" cevabi gunceller.

1.3 Metin Siniflandirmasinda Naive Bayes

Temel Fikir

Bir yorum geldiginde su oran hesaplanir:

$$\frac{P(+|\text{metin})}{P(-|\text{metin})} = \frac{P(\text{metin}|+) \cdot P(+)}{P(\text{metin}|-) \cdot P(-)}$$

Paydaki $P(\text{metin})$ her iki tarafta ayni oldugu icin iptal olur!

"Naive" (Saf) Varsayim: Bag of Words

Her kelime birbirinden tamamen bagimsizdir varsayilir.

Bu nedenle $P(\text{metin}|\text{sinif}) = \text{kelimelerin ayri ayri olasilik carpimina esittir:}$

$$P(\text{metin}|+) = P(k_1|+) \cdot P(k_2|+) \cdot P(k_3|+) \dots$$

Bag of Words: Metin bir "torba" gibi dusunulur — kelimeler sayilir, sira kaybolur.

"Kral kraligceyi sever" = "Kralige krali sever" (kelime sirasi farketmez!)

Laplace (Add-1) Duzeltmesi

Problem: Egitim setinde hic gorulmemis bir kelime varsa $P = 0$ ve tum carpim sifirlanir!

Cozum: Her kelime sayisina 1 ekle (Smoothing):

$$P(\text{kelime}|\text{sinif}) = \frac{\text{count}(\text{kelime}, \text{sinif}) + 1}{\text{count}(\text{tumkelimeler}, \text{sinif}) + |V|}$$

- $|V|$ = Vocabulary = tum benzersiz kelime sayisi
- Hic gozukmeyen kelime artik 0 degil, kucuk bir pozitif deger alir

```
In [3]: # — Ders Slaytindaki Ornek: Adim Adim Cozum —————
#
# SENARYO (Jurafsky & Martin'den, slayta alinmis):
#
# Egitim seti:
#     NEGATIF: "just plain boring"
#             "entirely predictable and lacks energy"
#             "no surprises and very few laughs"
#     POZITIF: "very powerful"
#             "the most fun film of the summer"
#
```

```

# Test: "predictable with no fun"
# Soru: Bu metin POZiTİF mi NEGATİF mi?
#


---



egitim_metinleri = [
    'just plain boring',
    'entirely predictable and lacks energy',
    'no surprises and very few laughs',
    'very powerful',
    'the most fun film of the summer'
]
etiketler = ['negatif', 'negatif', 'negatif', 'pozitif', 'pozitif']
test_metin = 'predictable with no fun'

# ADIM 1: Vocabulary olustur
tum_kelimeler = set()
for metin in egitim_metinleri:
    for kelime in metin.lower().split():
        tum_kelimeler.add(kelime)

V = len(tum_kelimeler)
print("=== ADIM 1: Vocabulary ===")
print(f"|V| = {V} benzersiz kelime")
print(f"Kelimeler: {sorted(tum_kelimeler)}")

# ADIM 2: Prior olasiliklar
n_toplam = len(etiketler)
n_pozitif = etiketler.count('pozitif')
n_negatif = etiketler.count('negatif')
P_poz = n_pozitif / n_toplam
P_neg = n_negatif / n_toplam

print(f"\n=== ADIM 2: Prior Olasiliklar ===")
print(f"P(+) = {n_pozitif}/{n_toplam} = {P_poz:.3f} (egitim setinde pozitif orani)")
print(f"P(-) = {n_negatif}/{n_toplam} = {P_neg:.3f} (egitim setinde negatif orani)")

```

=== ADIM 1: Vocabulary ===

|V| = 20 benzersiz kelime

Kelimeler: ['and', 'boring', 'energy', 'entirely', 'few', 'film', 'fun', 'just', 'lacks', 'laughs', 'most', 'no', 'of', 'plain', 'powerful', 'predictable', 'summer', 'surprises', 'the', 'very']

=== ADIM 2: Prior Olasiliklar ===

P(+) = 2/5 = 0.400 (egitim setinde pozitif orani)

P(-) = 3/5 = 0.600 (egitim setinde negatif orani)

In [4]: # — Laplace Duzeltmeli Likelihood Hesaplama —

```

def laplace_olasilik(kelime, sinif, metinler, etiketler, V):
    """
    Laplace (add-1) duzeltmesiyle P(kelime|sinif) hesaplar.

    Formül: (siniftaki_kelime_sayisi + 1) / (siniftaki_toplam_kelime + V)

    Neden +1?: Hic gorulmemis kelimedede P=0 ve tum carpim sifirlanir.
    Her kelimeye kucuk bir 'sahte sayi' ekleyerek bunu onleriz.
    """

```

```

Neden +|V|?: Normallestime saglansin diye paydaya da V eklenir.
"""
sinif_kelimeleri = []
for metin, etiket in zip(metinler, etiketler):
    if etiket == sinif:
        sinif_kelimeleri.extend(metin.lower().split())

kelime_sayisi = sinif_kelimeleri.count(kelime.lower())
toplam_kelime = len(sinif_kelimeleri)
olasilik = (kelime_sayisi + 1) / (toplam_kelime + V)
return olasilik, kelime_sayisi, toplam_kelime

# Her test kelimesi icin likelihood hesapla
test_kelimeleri = test_metin.lower().split()
print(f"Test metni : '{test_metin}'")
print(f"Kelimeler : {test_kelimeleri}")
print(f"Vocabulary : |V| = {V}\n")

print("=" * 72)
print(f"{'Kelime':<14} {'POZITIF':^27} {'NEGATIF':^27}")
print(f"{'':14} {'cnt+1 / top+V':^27} {'cnt+1 / top+V':^27}")
print("=" * 72)

skor_poz = P_poz
skor_neg = P_neg

for kelime in test_kelimeleri:
    p_poz, c_poz, t_poz = laplace_olasilik(kelime, 'pozitif', egitim_metinleri, et
    p_neg, c_neg, t_neg = laplace_olasilik(kelime, 'negatif', egitim_metinleri, et
    skor_poz *= p_poz
    skor_neg *= p_neg
    print(f"{'kelime}' "
          f"{'c_poz'+1 / {t_poz}+{V} = {p_poz:.4f} "
          f"{'c_neg'+1 / {t_neg}+{V} = {p_neg:.4f}")

print("=" * 72)
print(f"Prior P(sinif)      P(+) = {P_poz:.3f}                P(-) = {P_neg:.3f}")
print("=" * 72)
print(f"\nNihai skor P(+|metin) = {skor_poz:.4e}")
print(f"Nihai skor P(-|metin) = {skor_neg:.4e}")
print()

if skor_poz > skor_neg:
    print(f"TAHMIN: POZITIF")
else:
    print(f"TAHMIN: NEGATIF (skor_neg={skor_neg:.4e} > skor_poz={skor_poz:.4e})")
    print(" Neden? 'predictable' ve 'no' negatif sinifta daha sikti.")
    print(" 'fun' her iki sinifta da az gorundu, fark yaratmadi.")
    print(f" Slayttaki degerler: P(+)=3.2e-5 P(-)=6.1e-5 -> uymaktadır")

```

Test metni : 'predictable with no fun'
 Kelimeler : ['predictable', 'with', 'no', 'fun']
 Vocabulary : |V| = 20

Kelime	POZITIF		NEGATIF	
	cnt+1 / top+V		cnt+1 / top+V	
'predictable'	0+1 / 9+20 = 0.0345	1+1 / 14+20 = 0.0588		
'with'	0+1 / 9+20 = 0.0345	0+1 / 14+20 = 0.0294		
'no'	0+1 / 9+20 = 0.0345	1+1 / 14+20 = 0.0588		
'fun'	1+1 / 9+20 = 0.0690	0+1 / 14+20 = 0.0294		
Prior P(sınıf)	P(+) = 0.400		P(-) = 0.600	

Nihai skor $P(+|\text{metin}) = 1.1311e-06$
 Nihai skor $P(-|\text{metin}) = 1.7960e-06$

TAHMİN: NEGATIF (skor_neg=1.7960e-06 > skor_poz=1.1311e-06)
 Neden? 'predictable' ve 'no' negatif sınıfta daha sikti.
 'fun' her iki sınıfta da az gorundu, fark yaratmadi.
 Slayttaki degerler: $P(+)=3.2e-5$ $P(-)=6.1e-5$ -> uymaktadır

```
In [5]: # — Naive Bayes Zayif Yonleri: Canli Demo —————
# Ders slaytinda anlatilanlar 3 zayif yon: bagimsizilik, baglam kaybI, sarkazm.
# Burada kucuk bir Turkce veri setiyle bu sorunlari gozlemleyecegiz.

ornek_egitim = [
    'harika film cok guzel',      'mukemmel senaryo begendim',
    'berbat film hic guzel degil', 'kotu senaryo begenmedim',
    'cok iyi cok guzel',         'super harika begendim',
    'hic iyi degil berbat',      'gercekten kotu izlemedim',
]
ornek_etiket = ['poz', 'poz', 'neg', 'neg', 'poz', 'poz', 'neg', 'neg']

pipe_demo = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=50)),
    ('nb', MultinomialNB(alpha=1.0))
])
pipe_demo.fit(ornek_egitim, ornek_etiket)

# Test cumlecikleri
test_cumlecikler = {
    "Acik pozitif"      : ('harika guzel begendim',      'beklenen: POZ'),
    "Acik negatif"     : ('berbat kotu begenmedim',      'beklenen: NEG'),
    "Negasyon (slayt)" : ('hic fena degil',              'gercekte POZ ama...'),
    "Siralama farketmez" : ('guzel degil harika',         'ayni kelimeler, farkli'),
    "Sarkazm (slayt)"  : ('harika hizmetti 2 saat bekledim', 'gercekte NEG ama...'),
    "Karma baglam"     : ('guzel ama berbat degil',      'hem + hem - kelime var')
}

print("=== Naive Bayes Zayif Yonleri Demo ===")
print(f"{'Cumle':<45} {'Tahmin':^10} {'Not'}")
print("-" * 85)
```



```

for baslik, (cumle, not_str) in test_cumlecikler.items():
    tahmin = pipe_demo.predict([cumle])[0]
    simge = '[' + ']' if tahmin == 'poz' else '[' + ']'
    print(f"{cumle:<45} {simge} {tahmin:<10} <- {not_str}")

print()
print("TEMEL SORUN: Bag of Words kelime SAYAR, anlam ANLAMAZ.")
print(" 'harika' kelimesi her zaman pozitif agirlik tasir.")
print(" 'harika hizmetti 2 saat bekledim' sarkastik olsa da model yanliscik tahmin")
print(" Bu yuzden buyuk dil modelleri (BERT, GPT) baglamdan anlam cikararabilir.")

```

=== Naive Bayes Zayif Yonleri Demo ===

Cumle	Tahmin	Not
-		
harika guzel begendim	[+] poz	<- beklenen: POZ
berbat kotu begenmedim	[-] neg	<- beklenen: NEG
hic fena degil	[-] neg	<- gercekte POZ ama...
guzel degil harika	[+] poz	<- ayni kelimeler, fark
li anlam!		
harika hizmetti 2 saat bekledim	[+] poz	<- gercekte NEG ama...
guzel ama berbat degil	[-] neg	<- hem + hem - kelime v
ar		

TEMEL SORUN: Bag of Words kelime SAYAR, anlam ANLAMAZ.

'harika' kelimesi her zaman pozitif agirlik tasir.

'harika hizmetti 2 saat bekledim' sarkastik olsa da model yanliscik tahmin edebili

r.
Bu yuzden buyuk dil modelleri (BERT, GPT) baglamdan anlam cikararabilir.

1.4 Naive Bayes Varyantlari

scikit-learn'de uc farkli Naive Bayes implementasyonu vardir:

Varyant	Ne Zaman?	Veri Tipi	Ornek Kullanim
MultinomialNB	Metin siniflandirma	Kelime sayimlari / TF-IDF	Spam tespiti, haber kategorisi
GaussianNB	Sayisal oznitelikler	Surekli, normal dagilimli	Tibbi olcumler, sensor verisi
BernoulliNB	Ikili (var/yok) oznitelikler	0/1	Kelime var mi yok mu

Hangisini ne zaman?

- Metin uzerinde calisiliyorsa: **MultinomialNB** (kelime sayimlari negatif olamaz)
- Olcum/sayisal veri varsa: **GaussianNB** (surekli dagilim varsayimi)
- Her oznitelik "var/yok" seklindeyse: **BernoulliNB**

Ekstra — Makale bağlantısı:

Yahya et al. (2025), embedding vektorleri ile GaussianNB kullanıldığında yalnızca **%68** accuracy elde etti.

Neden? TF-IDF/embedding vektorleri Gaussian dağılım takip etmez.

MultinomialNB aynı metin görevinde çok daha iyidir.

```
In [6]: # Bu hücre internet erişimi gerektirir.
# Asagidaki kodu kendi ortamında internet varken çalıştırın.
# Notebook bu hücre olmadan da tam çalışır.
from sklearn.datasets import fetch_20newsgroups
# # — 20 Newsgroups: Gerçek Metin Sınıflandırma Projesi —————
# # 4 haber kategorisi: tip, uzay, beyzbol, silah politikası
# Pipeline = TF-IDF (metni sayıya çevir) + MultinomialNB (sınıflandır)#
kategoriler = ['sci.med', 'sci.space', 'rec.sport.baseball', 'talk.politics.guns']
print("Veri indiriliyor... (ilk seferinde birkaç saniye surebilir)")
egitim_ng = fetch_20newsgroups(subset='train', categories=kategoriler,
                              remove=('headers', 'footers', 'quotes'))
test_ng = fetch_20newsgroups(subset='test', categories=kategoriler,
                             remove=('headers', 'footers', 'quotes'))
print(f"Eğitim seti: {len(egitim_ng.data):,} haber")
print(f"Test seti : {len(test_ng.data):,} haber")
print(f"Kategoriler: {egitim_ng.target_names}")
print(f"\nÖrnek haber (ilk 300 karakter):")
print("-" * 50)
print(egitim_ng.data[0][:300])
print("-" * 50)
```

Veri indiriliyor... (ilk seferinde birkaç saniye surebilir)

Eğitim seti: 2,330 haber

Test seti : 1,551 haber

Kategoriler: ['rec.sport.baseball', 'sci.med', 'sci.space', 'talk.politics.guns']

Örnek haber (ilk 300 karakter):

It doesn't make a whole lot of difference, actually, since they weren't building spares of the station hardware anyway. (Dumb.) At least this is only one launch to fail.

```
In [7]: # — MultinomialNB Pipeline Eğitimi —————
#
# TF-IDF nedir?
#   TF = Term Frequency: Kelime bu belgede kaç kez geçti?
#   IDF = Inverse Document Frequency: Kelime tüm belgeler arasında ne kadar nadir?
#   TF-IDF = TF x IDF -> Yaygın kelimeler (the, and...) küçük ağırlık alır
#
# ngram_range=(1,2): Tek kelime + ikili kelime gruplarını da öznitelik yap
#   Örnek: 'home run', 'gun control', 'second amendment' gibi ikilileri de yakalar.
# min_df=1: Küçük veri setinde en az 1 belgede geçen kelimeleri al.
# Eğitim verilerini değişkene atama
egitim_ng_metinler = eğitim_ng.data
egitim_ng_etiketler = eğitim_ng.target
```

```
# Test verilerini deđışkene atama
test_ng_metinler = test_ng.data
test_ng_etiketler = test_ng.target
nb_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        max_features=500,
        ngram_range=(1, 2),
        min_df=1
    )),
    ('clf', MultinomialNB(alpha=1.0)) # alpha=1 = Laplace duzeltmesi
])

nb_pipeline.fit(egitim_ng_metinler, egitim_ng_etiketler)
y_tahmin_nb = nb_pipeline.predict(test_ng_metinler)
nb_acc = accuracy_score(test_ng_etiketler, y_tahmin_nb)

print(f"MultinomialNB Accuracy: {nb_acc:.4f} ({nb_acc*100:.1f}%)\\n")
print(classification_report(test_ng_etiketler, y_tahmin_nb,
                             target_names=kategoriler))
```

MultinomialNB Accuracy: 0.7389 (73.9%)

	precision	recall	f1-score	support
sci.med	0.72	0.81	0.76	397
sci.space	0.70	0.77	0.73	396
rec.sport.baseball	0.79	0.68	0.73	394
talk.politics.guns	0.77	0.68	0.72	364
accuracy			0.74	1551
macro avg	0.74	0.74	0.74	1551
weighted avg	0.74	0.74	0.74	1551

In [8]: # — Confusion Matrix + Her Kategorinin En Ayirt Edici Kelimeleri —————

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Sol: Confusion Matrix
cm_ng = confusion_matrix(test_ng_etiketler, y_tahmin_nb)
kisa_isimler = ['Tip', 'Uzay', 'Beyzbol', 'Silah']

sns.heatmap(cm_ng, annot=True, fmt='d', cmap='Blues', ax=ax1,
             xticklabels=kisa_isimler, yticklabels=kisa_isimler,
             linewidths=0.5, linecolor='white', annot_kws={'size': 12})
ax1.set_title(f'Naive Bayes: Confusion Matrix\\nAccuracy: {nb_acc:.3f}',
              fontsize=12, fontweight='bold')
ax1.set_xlabel('Tahmin Edilen', fontsize=11)
ax1.set_ylabel('Gercek Sinif', fontsize=11)

# Sag: Her kategorinin en ayirt edici 8 kelimesi
ozellikler_ng = nb_pipeline.named_steps['tfidf'].get_feature_names_out()
log_olasilik_ng = nb_pipeline.named_steps['clf'].feature_log_prob_
n_top = 8
renkler_ng = ['#3498DB', '#27AE60', '#E67E22', '#E74C3C']
```

```

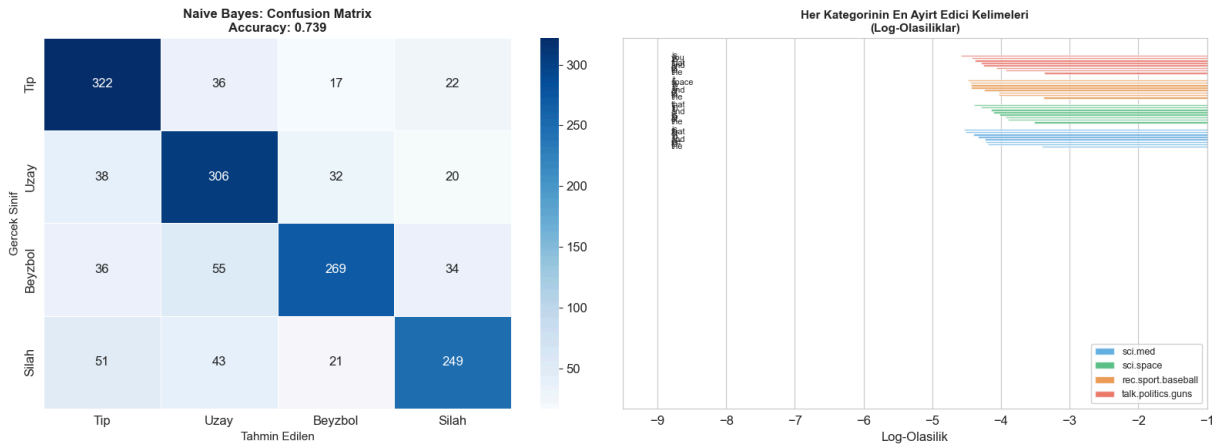
for i, (kat, renk) in enumerate(zip(kategoriler, renkler_ng)):
    top_idx = np.argsort(log_olasilik_ng[i])[-n_top:]
    top_words = [ozellikler_ng[j] for j in top_idx[::-1]]
    y_pos = [i * (n_top + 2) + j for j in range(n_top)]
    ax2.barh(y_pos, log_olasilik_ng[i][top_idx[::-1]], color=renk, alpha=0.75)
    for j, kelime in enumerate(top_words):
        ax2.text(-8.8, y_pos[j], kelime, va='center', fontsize=8)

ax2.set_title('Her Kategorinin En Ayirt Edici Kelimeleri\n(Log-Olasiliklar)',
              fontsize=11, fontweight='bold')
ax2.set_xlabel('Log-Olasilik')
ax2.set_yticks([])
ax2.set_xlim(-9.5, -1)

# Legend
for renk, kat in zip(renkler_ng, kategoriler):
    ax2.barh([-99], [0], color=renk, alpha=0.75, label=kat)
ax2.legend(loc='lower right', fontsize=9)

plt.tight_layout()
plt.savefig('nb_analiz.png', dpi=120, bbox_inches='tight')
plt.show()
print("Modelin ogrendigi kategorilere ozgu kelimeler gozlemleniyor.")
print("Gercek veri setinde bu kelimeler cok daha anlamlı olacaktır.")

```



Modelin ogrendigi kategorilere ozgu kelimeler gozlemleniyor.
Gercek veri setinde bu kelimeler cok daha anlamlı olacaktır.

BOLUM 2: KARAR AGACLARI & RANDOM FOREST

2.1 Karar Agaci Nedir?

Karar agaci, veriyi **"evet/hayir"** sorulari sorarak bolen bir algoritmadir.
Hem **siniflandirma** hem **regresyon** problemlerinde kullanilabilir.

Yapi:

- **Kok Dugum (Root):** En bilgilendirici ilk bolme
- **lc Dugumler (Nodes):** Ara bolme sorulari
- **Yapraklar (Leaves):** Son karar / sinif etiketi

Agac nasil buyur?

Her dugumde: "Hangi ozelligi hangi esik degerinde bolsem, alt gruplar en homojen (tek tip) olur?"

Durma kosullari (ozyinelemeli bolme su kosulda durur):

- Tumdugumler ayni sinifa aittir (Gini = 0, tam saf)
- Kullanilacak oznitelik kalmadi
- max_depth gibi bir hiperparametre sinirlanmasi yapildi

2.2 Bolme Kriteri: Gini Safsizligi

Hangi bolmenin "en iyi" olduguna **Gini safsizligi** ile karar verilir:

$$\text{Gini} = 1 - \sum_{k=1}^K p_k^2$$

- Gini = 0 → Mukemmel saf dugum (tek sinif) — ideal
- Gini = 0.5 → Maksimum karisiklik (2 sinif, 50-50 dagilim) — en kotu

Secim kurali: Her olasi bolme icin Gini hesaplanir.

En cok azalan (en saf alt gruplari ureten) bolme secilir.

Buna **Information Gain** (Bilgi Kazanimi) denir.

```
In [9]: import numpy as np
import matplotlib.pyplot as plt

# — Gini Safsizligi: Sezgiyi Gorsellestir —————

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# Sol: Gini egrisi
p = np.linspace(0, 1, 200)
gini_2sinif = 2 * p * (1 - p)

ax1.plot(p, gini_2sinif, color='#E74C3C', linewidth=2.5)
ax1.fill_between(p, gini_2sinif, alpha=0.12, color='#E74C3C')
ax1.scatter([0, 0.5, 1], [0, 0.5, 0], s=100, zorder=5, color='#E74C3C', edgecolors=

ax1.annotate('Gini=0: Tek sinif (tam saf)',
             xy=(0, 0), xytext=(0.08, 0.35),
             fontsize=9, color='#27AE60', fontweight='bold',
             arrowprops=dict(arrowstyle='->', color='#27AE60', lw=1.5))
ax1.annotate('Gini=0.5: 50-50 (en kotu)',
```

```

        xy=(0.5, 0.5), xytext=(0.58, 0.38),
        fontsize=9, color='#E74C3C', fontweight='bold',
        arrowprops=dict(arrowstyle='->', color='#E74C3C', lw=1.5))
ax1.annotate('Gini=0: Tek sinif (tam saf)',
            xy=(1, 0), xytext=(0.65, 0.22),
            fontsize=9, color='#27AE60', fontweight='bold',
            arrowprops=dict(arrowstyle='->', color='#27AE60', lw=1.5))

ax1.set_xlabel('p (Pozitif sinif oranı)', fontsize=12)
ax1.set_ylabel('Gini Safsizligi', fontsize=12)
ax1.set_title('Gini Safsizligi: 2 Sinifli Durum\nBolme hedefi: Gini azalmasini maks
            fontsize=11, fontweight='bold')
ax1.grid(True, alpha=0.3)
ax1.set_ylim(-0.02, 0.60)

# Sag: Bolme ornegi
ax2.axis('off')
ax2.set_title('Gini ile Bolme Secimi Ornegi', fontsize=12, fontweight='bold')

metin = """Kok dugumde 10 gozlem:
    5 pozitif, 5 negatif
    Gini_kok = 1-(0.5^2+0.5^2) = 0.50

Secen A: 'yas < 30' bolmesi
    Sol: 4 poz, 1 neg -> Gini = 0.32
    Sag: 1 poz, 4 neg -> Gini = 0.32
    Agirlikli ort.      = 0.32
    Gini Azalmasi      = 0.50 - 0.32 = 0.18 (iyi)

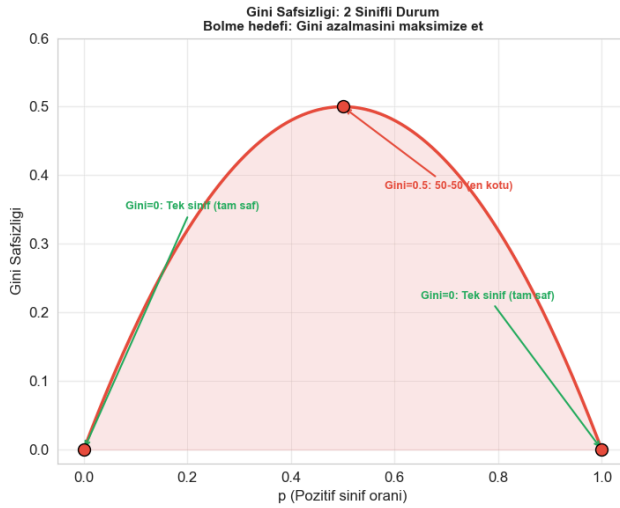
Secen B: 'gelir > 50K' bolmesi
    Sol: 3 poz, 3 neg -> Gini = 0.50
    Sag: 2 poz, 2 neg -> Gini = 0.50
    Agirlikli ort.      = 0.50
    Gini Azalmasi      = 0.50 - 0.50 = 0.00 (kotu)

KARAR: Secen A secilir!
    Daha saf alt gruplar urettiginden."""

ax2.text(0.05, 0.95, metin, transform=ax2.transAxes,
        fontsize=11, verticalalignment='top', family='monospace',
        bbox=dict(boxstyle='round,pad=0.7', facecolor='#F8F9FA', alpha=0.9))

plt.tight_layout()
plt.savefig('gini_visualize.png', dpi=120, bbox_inches='tight')
plt.show()

```



Gini ile Bolme Secimi Ornegi

Kok dugunde 10 gozlem:
5 pozitif, 5 negatif
 $Gini_{kok} = 1 - (0.5^2 + 0.5^2) = 0.50$

Secen A: 'yas < 30' bolmesi
Sol: 4 poz, 1 neg -> Gini = 0.32
Sag: 1 poz, 4 neg -> Gini = 0.32
Agirlikli ort. = 0.32
Gini Azalması = $0.50 - 0.32 = 0.18$ (iyi)

Secen B: 'gelir > 50K' bolmesi
Sol: 3 poz, 3 neg -> Gini = 0.50
Sag: 2 poz, 2 neg -> Gini = 0.50
Agirlikli ort. = 0.50
Gini Azalması = $0.50 - 0.50 = 0.00$ (kötü)

KARAR: Secen A secilir!
Daha saf alt gruplar urettiginden.

```
In [10]: # — Farkli Derinliklerde Karar Agaci Performansi —————
# Wine veri seti: 3 sarap tipi, 13 kimyasal oznitelik
# Derinlik arttikca ne oldugunu izleyelim.

wine = load_wine()
X_w, y_w = wine.data, wine.target
X_w_tr, X_w_te, y_w_tr, y_w_te = train_test_split(X_w, y_w, test_size=0.25, random_

print("=== Farkli Derinliklerde Karar Agaci ===")
print(f"{'max_depth':<12} {'Egitim Acc':>12} {'Test Acc':>10} {'Yaprak Sayisi':>14}")
print("-" * 52)

derinlik_sonuc_lari = {}
for derinlik in [1, 2, 3, 5, 10, None]:
    dt = DecisionTreeClassifier(max_depth=derinlik, random_state=42, criterion='gini')
    dt.fit(X_w_tr, y_w_tr)
    e_acc = dt.score(X_w_tr, y_w_tr)
    t_acc = dt.score(X_w_te, y_w_te)
    n_yap = dt.get_n_leaves()
    label = str(derinlik) if derinlik else 'Sinirsiz'
    derinlik_sonuc_lari[label] = {'egitim': e_acc, 'test': t_acc, 'yaprak': n_yap}
    isaretci = ' <- Overfitting!' if (e_acc - t_acc) > 0.05 else ''
    print(f"{'label':<12} {'e_acc':>12.3f} {'t_acc':>10.3f} {'n_yap':>14}{isaretci}")

print()
print("Gozlem: Derinlik artinca egitim accuracy 1.0'e gider,")
print("fakat test accuracy dusebilir -> OVERFITTING!")
print("Model egitim verisini 'ezberler', yeni veriye generalize edemez.")
```

=== Farkli Derinliklerde Karar Agaci ===

max_depth	Egitim Acc	Test Acc	Yaprak Sayisi
1	0.662	0.600	2 <- Overfitting!
2	0.940	0.867	4 <- Overfitting!
3	0.992	0.956	6
5	1.000	0.956	7
10	1.000	0.956	7
Sinirsiz	1.000	0.956	7

Gozlem: Derinlik artinca egitim accuracy 1.0'e gider,

fakat test accuracy dusebiir -> OVERFITTING!

Model egitim verisini 'ezberler', yeni veriye generalize edemez.

```
In [11]: # — Karar Agacini Ciz —————
# Karar agaclarinin en buyuk avantajı: insan tarafından okunabilir!

fig, axes = plt.subplots(1, 2, figsize=(18, 8))

for ax, derinlik, baslik_renk, yorum in [
    (axes[0], 3, '#27AE60', 'Iyi Denge'),
    (axes[1], None, '#E74C3C', 'Overfitting!')
]:
    dt_gorseli = DecisionTreeClassifier(max_depth=derinlik, random_state=42)
    dt_gorseli.fit(X_w_tr, y_w_tr)
    acc = dt_gorseli.score(X_w_te, y_w_te)

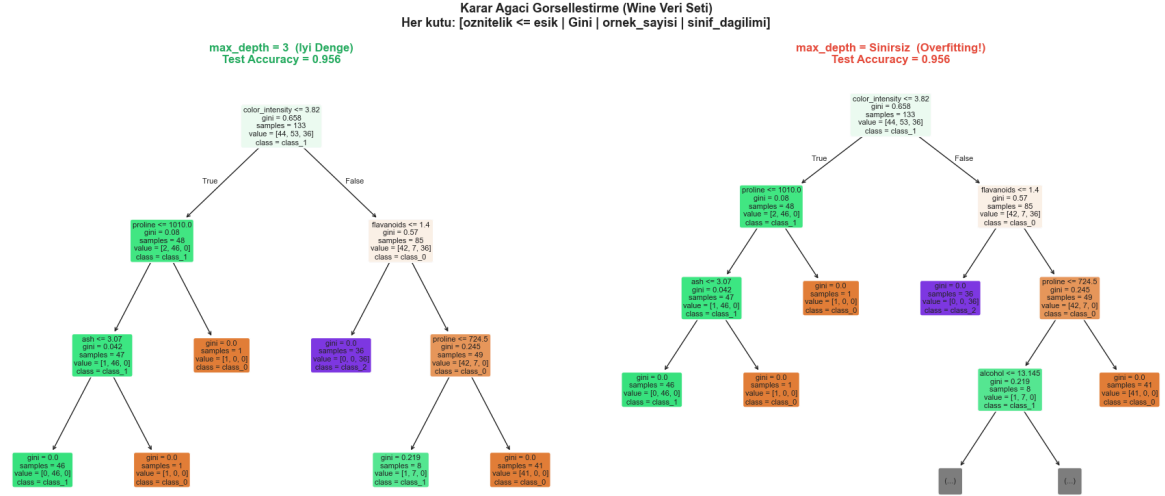
    plot_tree(
        dt_gorseli,
        feature_names=wine.feature_names,
        class_names=wine.target_names,
        filled=True,
        rounded=True,
        max_depth=3,
        fontsize=8,
        ax=ax,
        impurity=True
    )
    label = str(derinlik) if derinlik else 'Sinirsiz'
    ax.set_title(
        f'max_depth = {label} ({yorum})\nTest Accuracy = {acc:.3f}',
        fontsize=12, fontweight='bold', color=baslik_renk
    )

plt.suptitle(
    'Karar Agaci Gorsellestirme (Wine Veri Seti)\n'
    'Her kutu: [oznitelik <= esik | Gini | ornek_sayisi | sinif_dagilimi]',
    fontsize=13, fontweight='bold'
)
plt.tight_layout()
plt.savefig('karar_agaci_gorseli.png', dpi=100, bbox_inches='tight')
plt.show()

print("Kutu icerigi aciklamasi:")
print("  oznitelik <= esik : Bolme kurali (ornek: proline <= 755)")
print("  gini                : Dugumun safsizligi (0 = tam saf = iyi)")
```



```
print(" samples : Bu dugume dusen ornek sayisi")
print(" value : [sinif0_sayisi, sinif1_sayisi, sinif2_sayisi]")
```



Kutu icerigi aciklamasi:

oznitelik <= esik : Bolme kurali (ornek: proline <= 755)
gini : Dugumun safsizligi (0 = tam saf = iyi)
samples : Bu dugume dusen ornek sayisi
value : [sinif0_sayisi, sinif1_sayisi, sinif2_sayisi]

2.3 Overfitting'i Onlemek: Iki Strateji

Strateji 1: Budama (Pruning)

Yontem	Aciklama	scikit-learn parametresi
Erken Budama (pre-pruning)	Agac buyurken durdur	max_depth , min_samples_split , min_samples_leaf
Gec Budama (post-pruning)	Tam buyut, sonra dal kes	ccp_alpha (cost-complexity pruning)

Strateji 2: Ensemble (Topluluk) Yontemleri

"Bir agac yanlabilir ama orman yanilmaz."

Random Forest = Bagging + Rastgele Oznitelik Secimi

Nasil calisir?

- Bootstrap ornekleme:** Her agac icin egitim verisinden rastgele orneklem al (yerine koyarak)
- Rastgele oznitelik:** Her bolmede tum oznitelikler degil, rastgele bir alt kume kullanilir ($\sqrt{n_{\text{features}}}$)
- Cogunluk oylamasi:** Siniflandirmada en cok oy alan sinif secilir

Bu iki mekanizma agaclari birbirinden **bagimsiz** yapar, hatalar ortalamada iptal olur!

Ekstra — Bagging nedir?

Bootstrap Aggregating: Her agac verideki farkli "gorunumu" ogrenir, toplamda daha sagkam bir model olusur.

Random Forest bunun uzerine oznitelik rastgeleligini ekler, agaclar daha az korelasyonlu hale gelir -> daha guclu ensemble.

```
In [12]: # — Random Forest: Agac Sayisi Artinca Ne Olur? —————
# Bu grafik Random Forest'in temel prensibini gosteriyor:
# Agac sayisi arttikca test accuracy stabillesir ve overfitting azalir.

kanser = load_breast_cancer()
Xk_tr, Xk_te, yk_tr, yk_te = train_test_split(
    kanser.data, kanser.target, test_size=0.2, random_state=42
)
scaler = StandardScaler()
Xk_tr_s = scaler.fit_transform(Xk_tr)
Xk_te_s = scaler.transform(Xk_te)

agac_sayilari = [1, 3, 5, 10, 20, 50, 100, 200]
egitim_accler, test_accler = [], []

for n in agac_sayilari:
    rf = RandomForestClassifier(n_estimators=n, random_state=42, n_jobs=1)
    rf.fit(Xk_tr_s, yk_tr)
    egitim_accler.append(rf.score(Xk_tr_s, yk_tr))
    test_accler.append(rf.score(Xk_te_s, yk_te))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# Sol: Accuracy egrisi
ax1.plot(agac_sayilari, egitim_accler, 'b-o', label='Egitim Accuracy', linewidth=2,
ax1.plot(agac_sayilari, test_accler, 'r-s', label='Test Accuracy', linewidth=2,

# Tek agac vs 100 agac farki
idx_1 = agac_sayilari.index(1)
idx_100 = agac_sayilari.index(100)
ax1.annotate(f'Test={test_accler[idx_1]:.3f}\n(1 agac)',
             xy=(1, test_accler[idx_1]), xytext=(8, 0.925),
             fontsize=9, color='#E74C3C',
             arrowprops=dict(arrowstyle='->', color='#E74C3C'))
ax1.annotate(f'Test={test_accler[idx_100]:.3f}\n(100 agac)',
             xy=(100, test_accler[idx_100]), xytext=(60, 0.958),
             fontsize=9, color='#27AE60',
             arrowprops=dict(arrowstyle='->', color='#27AE60'))

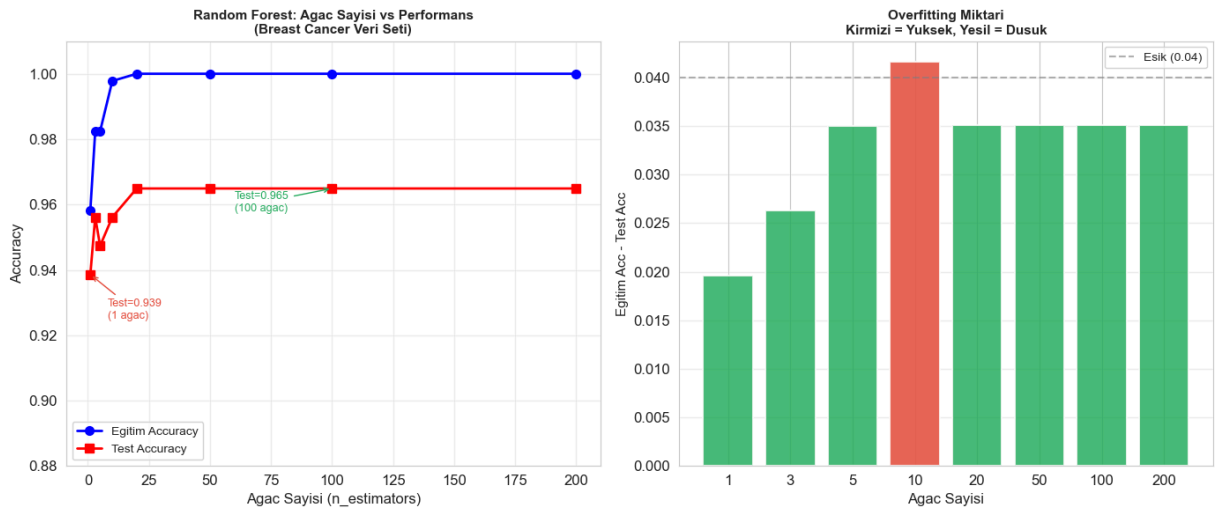
ax1.set_xlabel('Agac Sayisi (n_estimators)', fontsize=12)
ax1.set_ylabel('Accuracy', fontsize=12)
ax1.set_title('Random Forest: Agac Sayisi vs Performans\n(Breast Cancer Veri Seti)'
              fontsize=11, fontweight='bold')
ax1.legend(fontsize=10)
ax1.grid(True, alpha=0.3)
ax1.set_ylim([0.88, 1.01])
```

```
# Sag: Overfitting miktari (egitim - test farki)
fark = [e - t for e, t in zip(egitim_accler, test_accler)]
bar_renkleri = ['#E74C3C' if f > 0.04 else '#27AE60' for f in fark]

ax2.bar(range(len(agac_sayilari)), fark, color=bar_renkleri, alpha=0.85, edgecolor=
ax2.axhline(y=0.04, color='gray', linestyle='--', alpha=0.6, label='Esik (0.04)')
ax2.set_xticks(range(len(agac_sayilari)))
ax2.set_xticklabels(agac_sayilari)
ax2.set_xlabel('Agac Sayisi', fontsize=12)
ax2.set_ylabel('Egitim Acc - Test Acc', fontsize=11)
ax2.set_title('Overfitting Miktari\nKirmizi = Yuksek, Yesil = Dusuk', fontsize=11,
ax2.legend(fontsize=10)
ax2.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('rf_agac_sayisi.png', dpi=120, bbox_inches='tight')
plt.show()

print(f"En iyi test accuracy: {max(test_accler):.4f} ({agac_sayilari[test_accler.in
print("Gozlem: Agac sayisi artinca overfitting (egitim-test farki) azaliyor.")
```



En iyi test accuracy: 0.9649 (20 agacta)

Gozlem: Agac sayisi artinca overfitting (egitim-test farki) azaliyor.

```
In [13]: # — Feature Importance: Model Neye Bakiyor? —————
# Random Forest'in onemli yorumlanabilirlik ozelligi:
# Her oznitelegin model karariana katkisini olcebiliriz.
#
# Mean Decrease in Impurity (MDI):
# Her bolmede ortalama Gini azalmasina o ozniteligin katkisi.
# Yuksek skor = Model bu oznitelige cok guveniy = Onemli oznitelik

rf_final = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=1)
rf_final.fit(Xk_tr_s, yk_tr)

onem = rf_final.feature_importances_
ozellikler_k = kanser.feature_names
siralam = np.argsort(onem) # kucukten buyuge sirala (barh icin)

fig, ax = plt.subplots(figsize=(10, 9))
```

```

renkler_onem = plt.cm.RdYlGn(onem[siralama] / onem.max())
bars = ax.barh(range(len(siralama)), onem[siralama],
               color=renkler_onem, alpha=0.85, edgecolor='white')

ax.set_yticks(range(len(siralama)))
ax.set_yticklabels([ozellikler_k[i] for i in siralama], fontsize=9)

# Degerleri bar sonuna yaz
for i, (bar, val) in enumerate(zip(bars, onem[siralama])):
    ax.text(val + 0.001, i, f'{val:.3f}', va='center', fontsize=8, color='#2C3E50')

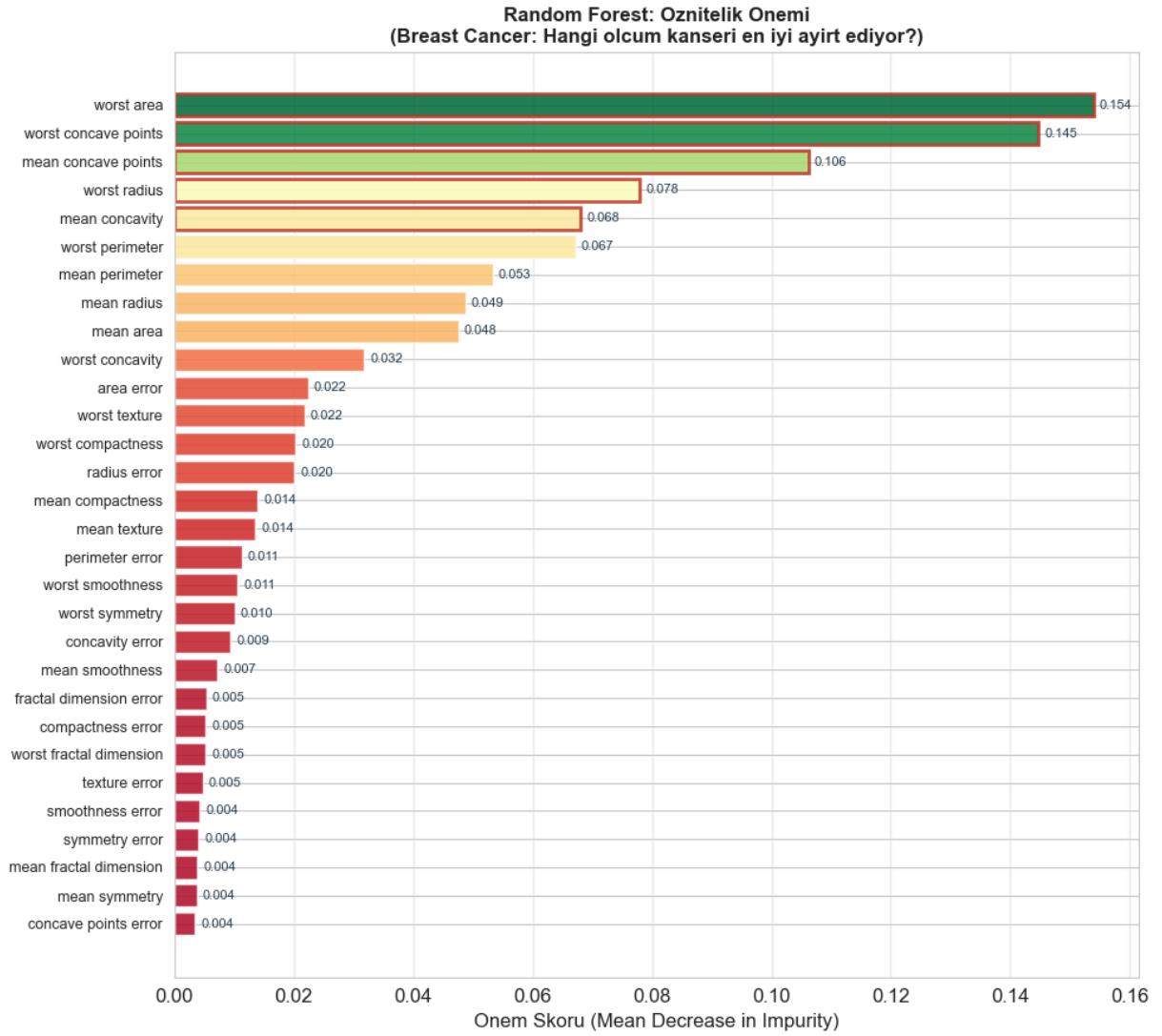
# En onemli 5'i vurgula
for i in range(len(siralama) - 5, len(siralama)):
    bars[i].set_edgecolor('#C0392B')
    bars[i].set_linewidth(2)

ax.set_xlabel('Onem Skoru (Mean Decrease in Impurity)', fontsize=12)
ax.set_title('Random Forest: Oznitelik Onemi\n'
             '(Breast Cancer: Hangi olcum kanseri en iyi ayirt ediyor?)',
             fontsize=12, fontweight='bold')
ax.grid(True, alpha=0.3, axis='x')

plt.tight_layout()
plt.savefig('rf_feature_importance.png', dpi=120, bbox_inches='tight')
plt.show()

print("En onemli 5 oznitelik (kirmizi kenarli):")
for i in range(len(siralama)-1, len(siralama)-6, -1):
    idx = siralama[i]
    print(f" {ozellikler_k[idx]:<35} skor: {onem[idx]:.4f}")
print()
print("Klinik anlami: Bu olcumler kanser tanisinda en ayirt edici biyo-belirtecler.")

```



En onemli 5 oznitelik (kirmizi kenarli):

worst area	skor: 0.1539
worst concave points	skor: 0.1447
mean concave points	skor: 0.1062
worst radius	skor: 0.0780
mean concavity	skor: 0.0680

Klinik anlami: Bu olcumler kanser tanisinda en ayirt edici biyo-belirtecler.

BOLUM 3: DEGERLENDIRME METRIKLERI (Evaluation Metrics)

3.1 Confusion Matrix (Karisiklik Matrisi)

Model bir tahmin yaptiginda dort sonuc mumkundur:

GERCEK DURUM	
Pozitif	Negatif

TAHMIN	Poz.	TP (Dogru +)	FP (Yanlis+)	Model "pozitif" dedi
	Neg.	FN (Yanlis-)	TN (Dogru -)	Model "negatif" dedi

Kisaltma	Acilim	Anlami
TP	True Positive	Pozitif, dogru tahmin edildi
TN	True Negative	Negatif, dogru tahmin edildi
FP	False Positive	Negatif ama pozitif dendi (Tip I Hata)
FN	False Negative	Pozitif ama negatif dendi (Tip II Hata)

3.2 Dort Temel Metrik

Accuracy — Genel dogrluk orani:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision — "Pozitif dedigimizin ne kadari gercekten pozitif?":

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensitivity) — "Gercek pozitiflerin ne kadarini yakaladik?":

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score — Precision ve Recall'un harmonik ortalamasi:

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.3 Hangi Metrigi Ne Zaman Kullanmeli?

Senaryo A — Kanser Taramasi:

FN (kanseri kacirmak) hayati tehdit eder! **Recall maksimize edilmeli.**

Bedeli: Precision dusur = daha fazla gereksiz biyopsi (FP artar ama kabul edilebilir)

Senaryo B — Spam Filtresi:

FP (onemli maili spam'e gondermek) kabul edilemez. **Precision maksimize edilmeli.**

Bedeli: Bazi spam'ler gelen kutusuna gecer (FN artar ama kabul edilebilir)

Senaryo C — Dengeli onem:

Ikisi de onemli -> **F1-Score**. Ozellikle dengesiz veri setlerinde.

Ekstra — Accuracy Paradoksu:

%99 negatif %1 pozitif olan bir veri setinde,

"her seye negatif de" diyen model %99 accuracy alir!

Ama hic bir pozitif yakalanamaz. Bu yuzden dengesiz veride F1 veya AUC-ROC tercih edilir.

```
In [14]: import matplotlib.pyplot as plt

# — Metrikleri Adim Adim Gorsellestir —————

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 7))

# Ornek degerler: kanser taramasi
TP, TN, FP, FN = 45, 82, 8, 5

# Sol: Confusion Matrix kutusu
matris = [[TP, FP], [FN, TN]]
renkler_m = [['#A9DFBF', '#FADBD8'], ['#FDEBD0', '#A9DFBF']]
etiketler_m = [['TP\n(Dogru Pozitif)', 'FP\n(Yanlis Pozitif)'],
               ['FN\n(Yanlis Negatif)', 'TN\n(Dogru Negatif)']]

for i in range(2):
    for j in range(2):
        ax1.add_patch(plt.Rectangle(
            (j, 1-i), 1, 1,
            facecolor=renkler_m[i][j], edgecolor='white', linewidth=3
        ))
        ax1.text(j+0.5, 1-i+0.65, str(matris[i][j]),
                 ha='center', va='center', fontsize=22, fontweight='bold', color='#555')
        ax1.text(j+0.5, 1-i+0.28, etiketler_m[i][j],
                 ha='center', va='center', fontsize=9, color='#555', style='italic')

ax1.set_xlim(0, 2); ax1.set_ylim(0, 2)
ax1.set_xticks([0.5, 1.5])
ax1.set_xticklabels(['Gercekte\nPozitif', 'Gercekte\nNegatif'], fontsize=11)
ax1.set_yticks([0.5, 1.5])
ax1.set_yticklabels(['Tahmin:\nNegatif', 'Tahmin:\nPozitif'], fontsize=11)
ax1.xaxis.set_label_position('top')
ax1.xaxis.tick_top()
ax1.set_title('Confusion Matrix Anatomisi\n(Kanser Taramasi Ornegi)', fontsize=12,

# Sag: Formül hesapları
ax2.axis('off')
ax2.set_title('Adim Adim Metrik Hesabi', fontsize=12, fontweight='bold')

acc = (TP + TN) / (TP + TN + FP + FN)
pre = TP / (TP + FP)
rec = TP / (TP + FN)
f1 = 2 * pre * rec / (pre + rec)

hesap_metni = f"TP={TP} TN={TN} FP={FP} FN={FN} (Toplam={TP+TN+FP+FN})"
```

```

ACCURACY: Genel dogruluk
(TP+TN)/Toplam = ({TP}+{TN})/({TP}+{TN}+{FP}+{FN}) = {acc:.3f}
Tahminlerin %{acc*100:.1f}'i dogru

PRECISION: Pozitif tahminlerin guvenilirlik
TP/(TP+FP) = {TP}/({TP}+{FP}) = {pre:.3f}
Kanser dedigimizin %{pre*100:.1f}'i gercekten kanser

RECALL: Gercek pozitifleri yakalama
TP/(TP+FN) = {TP}/({TP}+{FN}) = {rec:.3f}
Kanserlerin %{rec*100:.1f}'ini tespit ettik
{FN} hasta gozden kacti! (FN = hayati tehdit)

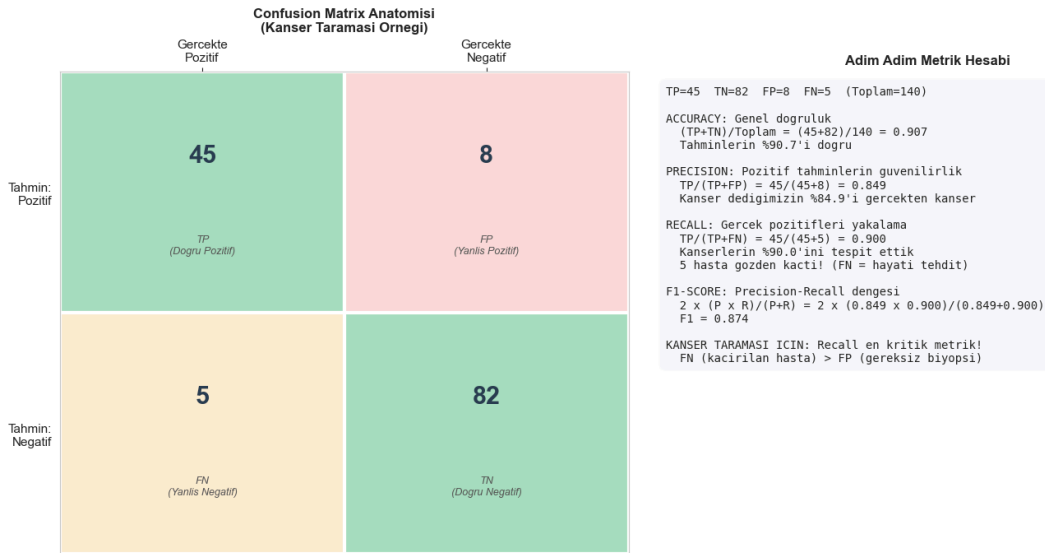
F1-SCORE: Precision-Recall dengesi
2 x (P x R)/(P+R) = 2 x ({pre:.3f} x {rec:.3f})/({pre:.3f}+{rec:.3f})
F1 = {f1:.3f}

KANSER TARAMASI ICIN: Recall en kritik metrik!
FN (kacirilan hasta) > FP (gereksiz biyopsi)"""

ax2.text(0.04, 0.97, hesap_metni, transform=ax2.transAxes,
        fontsize=10, verticalalignment='top', family='monospace',
        bbox=dict(boxstyle='round,pad=0.6', facecolor='#F8F9FA', alpha=0.95))

plt.tight_layout()
plt.savefig('metrikler_aciklama.png', dpi=120, bbox_inches='tight')
plt.show()

```



BOLUM 4: KAPSAMLI MODEL KARSILASTIRMASI

4.1 Senaryo

Gogus kanseri veri setinde **GaussianNB, Decision Tree ve Random Forest** modellerini ayni kosullarda karsilastiriyoruz.

Veri seti: 30 medikal oznitelik (hucre sekli, boyutu, dokusu olcumleri)
569 ornek: 212 malignant (kotu huylu), 357 benign (iyi huylu)

Onemli not: Veri seti dengesiz (%63 benign, %37 malignant).
Bu yuzden sadece accuracy yeterli degil, recall ozellikle onemli!

Makale baglantisi (Yahya et al., 2025):

Ayni karsilastirma Arapca haber metinlerinde yapildiginda:

RandomForest %93 | XGBoost %92 | Gaussian NB %68

Buyuk dil modelleri (BERT/GPT) bile geleneksel ML'in gerisinde kaldi!

Neden? Bu gorevde feature engineering + siniflandirma odakli modeller jeneratif modellerden (T5, GPT) cok daha verimli.

```
In [15]: # — Veri Hazirligi (Veri Sizintisi Kontrolu) —————
#
# Kritik Not: StandardScaler'i SADECE egitim setine fit etmek gerekir!
# Test setine fit edilirse, test bilgisi modele 'sizar' ve sonuclar yaniltici olur.
# Bu, makine ogrenmesindeki en yaygin hatalardan biridir.

kanser = load_breast_cancer()
Xk_tr, Xk_te, yk_tr, yk_te = train_test_split(
    kanser.data, kanser.target,
    test_size=0.2, random_state=42,
    stratify=kanser.target # Sinif oranini koru
)
scaler = StandardScaler()
Xk_tr_s = scaler.fit_transform(Xk_tr) # fit_transform: egitim verisinden ogrenip u
Xk_te_s = scaler.transform(Xk_te)     # sadece transform: egitimden ogrenen parametre

print(f"Egitim seti : {Xk_tr_s.shape[0]} ornek")
print(f"Test seti   : {Xk_te_s.shape[0]} ornek")
print(f"Oznitelik   : {Xk_tr_s.shape[1]}")
print(f"Malignant   : {(yk_te==0).sum()} test ornegi")
print(f"Benign      : {(yk_te==1).sum()} test ornegi")
```

```
Egitim seti : 455 ornek
Test seti   : 114 ornek
Oznitelik   : 30
Malignant   : 42 test ornegi
Benign      : 72 test ornegi
```

```
In [16]: # — Modelleri Egit ve Karsilastir —————

modeller_k = {
    'GaussianNB': GaussianNB(),
    'DecisionTree (depth=5)': DecisionTreeClassifier(max_depth=5, random_state=42),
    'DecisionTree (budanmamis)': DecisionTreeClassifier(random_state=42),
    'RandomForest (100 agac)': RandomForestClassifier(n_estimators=100, random_state=42)
}
```

```

karsilastirma = {}
print("=== Model Egitimi ve Degerlendirir ===")
print(f"{'Model':<30} {'Acc':>7} {'Prec':>7} {'Rec':>7} {'F1':>7}")
print("-" * 60)

for isim, model in modeller_k.items():
    model.fit(Xk_tr_s, yk_tr)
    y_p = model.predict(Xk_te_s)
    acc = accuracy_score(yk_te, y_p)
    pre = precision_score(yk_te, y_p)
    rec = recall_score(yk_te, y_p)
    f1 = f1_score(yk_te, y_p)
    karsilastirma[isim] = {'Accuracy': acc, 'Precision': pre, 'Recall': rec, 'F1': f1}
    isaretci = ' <- Overfitting?' if isim == 'DecisionTree (budanmamis)' else ''
    print(f"{'isim':<30} {'acc':>7.3f} {'pre':>7.3f} {'rec':>7.3f} {'f1':>7.3f}{isaretci}")

print()
print("Gozlem 1: Budanmamis DT en dusuk recall -> kanser hastasi kaciriyor!")
print("Gozlem 2: RandomForest dengeli ve yuksek performans sagliyor.")
print("Gozlem 3: GaussianNB basit ama hizli bir baseline olarak kabul edilebilir.")

```

```

=== Model Egitimi ve Degerlendirir ===

```

Model	Acc	Prec	Rec	F1	
GaussianNB	0.930	0.944	0.944	0.944	
DecisionTree (depth=5)	0.921	0.957	0.917	0.936	
DecisionTree (budanmamis)	0.912	0.956	0.903	0.929	<- Overfitting?
RandomForest (100 agac)	0.956	0.959	0.972	0.966	

Gozlem 1: Budanmamis DT en dusuk recall -> kanser hastasi kaciriyor!

Gozlem 2: RandomForest dengeli ve yuksek performans sagliyor.

Gozlem 3: GaussianNB basit ama hizli bir baseline olarak kabul edilebilir.

```

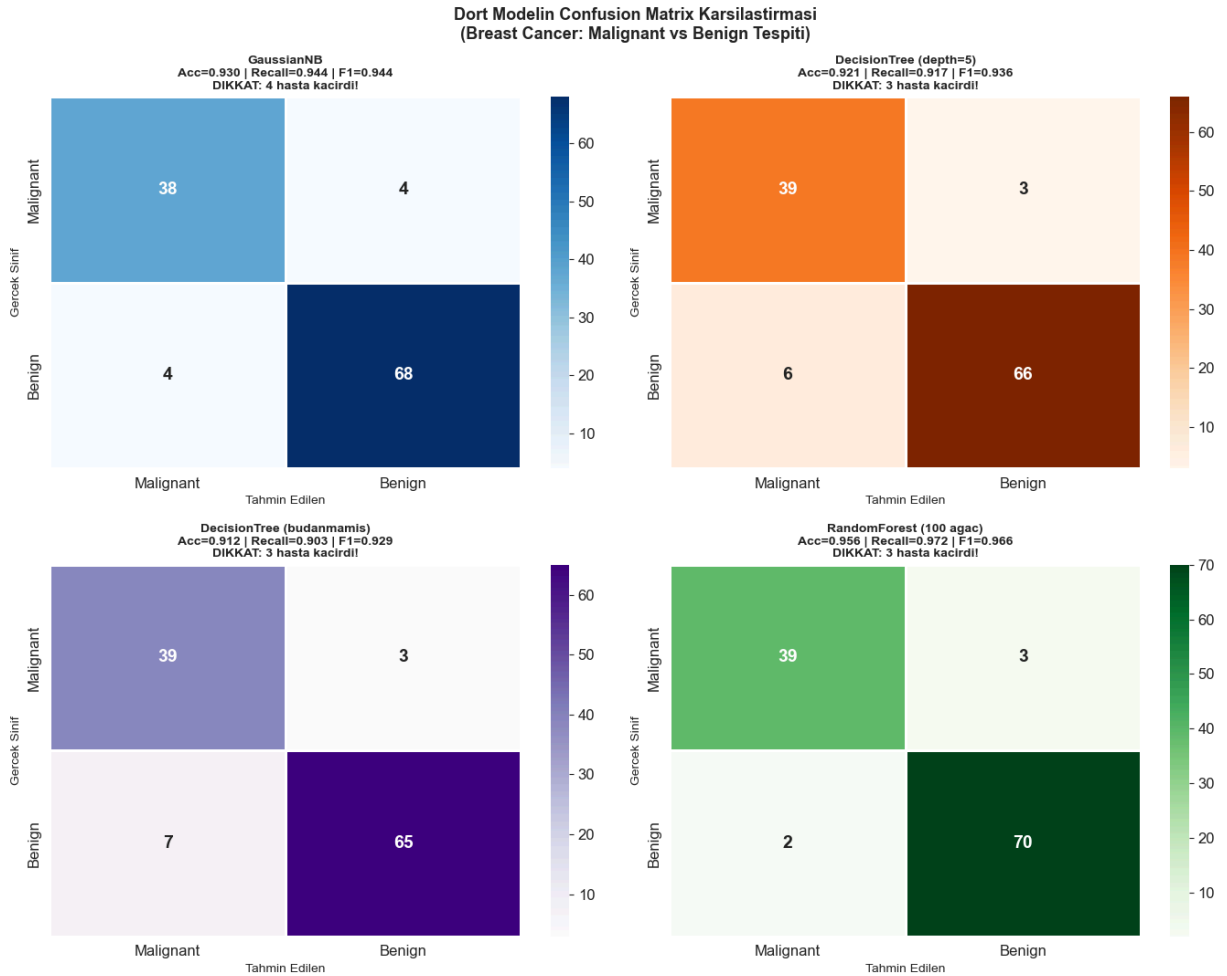
In [17]: # — Dort Confusion Matrix Yan Yana —————

fig, axes = plt.subplots(2, 2, figsize=(14, 11))
axes = axes.flatten()
cmaplar = ['Blues', 'Oranges', 'Purples', 'Greens']

for ax, (isim, sonuc), cmap in zip(axes, karsilastirma.items(), cmaplar):
    cm = confusion_matrix(yk_te, sonuc['y_p'])
    sns.heatmap(
        cm, annot=True, fmt='d', cmap=cmap, ax=ax,
        xticklabels=['Malignant', 'Benign'],
        yticklabels=['Malignant', 'Benign'],
        linewidths=1, linecolor='white',
        annot_kws={'size': 14, 'weight': 'bold'}
    )
    fn_val = cm[0][1]
    uyari = f'DIKKAT: {fn_val} hasta kacirdi!' if fn_val > 2 else f'{fn_val} hasta'
    ax.set_title(
        f'{isim}\nAcc={sonuc["Accuracy"]:.3f} | Recall={sonuc["Recall"]:.3f} | F1={sonuc["F1"]:.3f}'
        f'fontsize=10, fontweight=bold'
    )
    ax.set_xlabel('Tahmin Edilen', fontsize=10)
    ax.set_ylabel('Gercek Sinif', fontsize=10)

```

```
plt.suptitle(
    'Dort Modelin Confusion Matrix Karsilastirmasi\n'
    '(Breast Cancer: Malignant vs Benign Tespiti)',
    fontsize=13, fontweight='bold'
)
plt.tight_layout()
plt.savefig('model_confusion_karsilastirma.png', dpi=110, bbox_inches='tight')
plt.show()
```



```
In [18]: # — Radar Chart: Dort Metrik Birlikte —————
# Radar (Spider) chart ile 4 metrigi ayni anda gorsellestiriyoruz.
# Dis = Iyi, Ic = Kotu | Genis alan = Genel olarak daha iyi model

metrikler_r = ['Accuracy', 'Precision', 'Recall', 'F1']
n_m = len(metrikler_r)

# Acilar esit aralikli olarak olusturulur ve son = ilk ile kapatilir
acilar = np.linspace(0, 2 * np.pi, n_m, endpoint=False).tolist()
acilar += acilar[:1]

renk_map = {
    'GaussianNB': '#3498DB',
    'DecisionTree (depth=5)': '#E67E22',
    'DecisionTree (budanmamis)': '#E74C3C',
    'RandomForest (100 agac)': '#27AE60',
}
```

```

}

fig, ax = plt.subplots(figsize=(9, 9), subplot_kw=dict(polar=True))

for isim, sonuc in karsilastirma.items():
    degerler = [sonuc[m] for m in metrikler_r]
    degerler += degerler[:1] # kapat

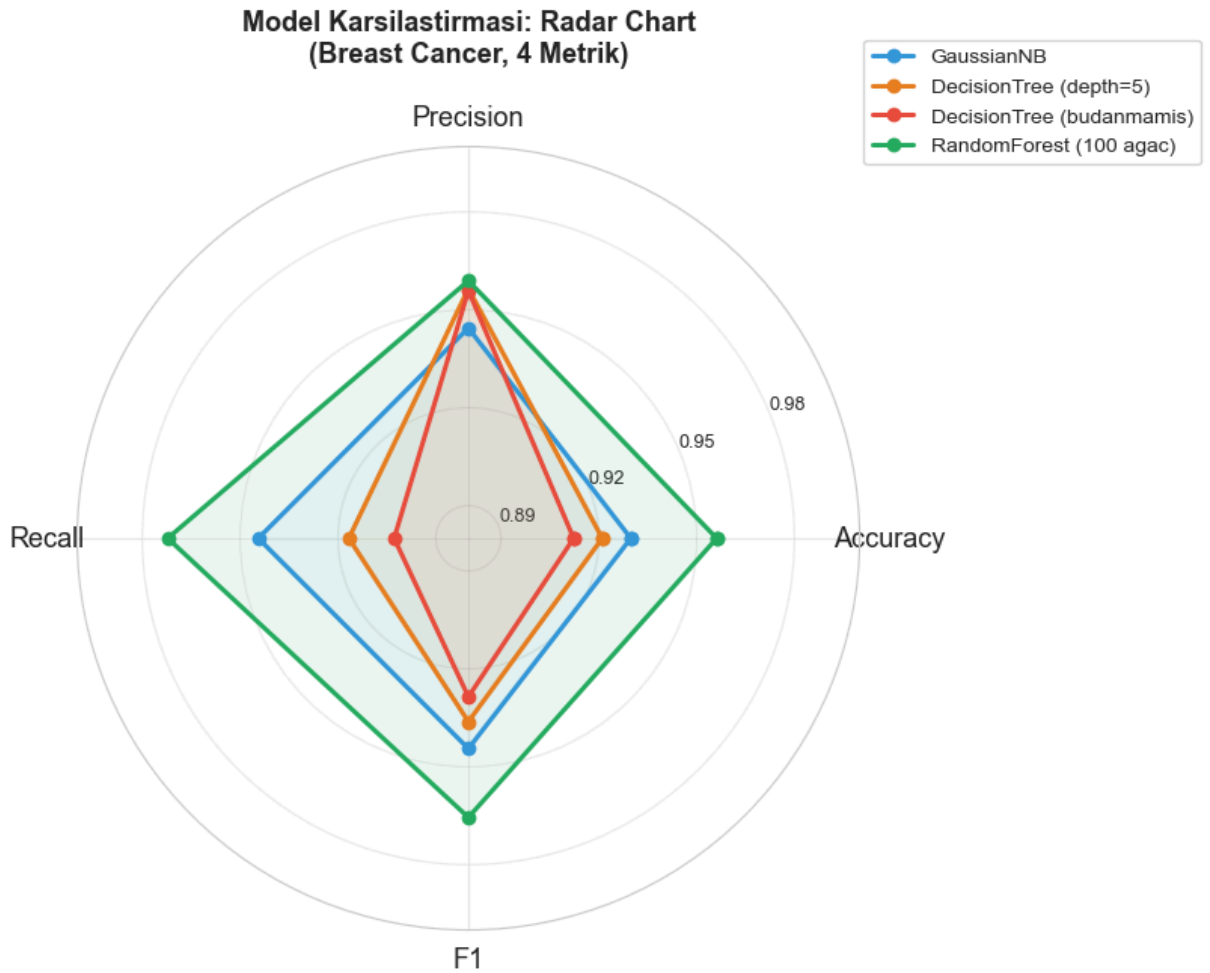
    ax.plot(acilar, degerler, 'o-', linewidth=2.2,
            label=isim, color=renk_map[isim])
    ax.fill(acilar, degerler, alpha=0.07, color=renk_map[isim])

ax.set_thetagrids(np.degrees(acilar[:-1]), metrikler_r, fontsize=13)
ax.set_ylim(0.88, 1.0)
ax.set_yticks([0.89, 0.92, 0.95, 0.98])
ax.set_yticklabels(['0.89', '0.92', '0.95', '0.98'], fontsize=9)
ax.set_title('Model Karsilastirmasi: Radar Chart\n(Breast Cancer, 4 Metrik)',
             fontsize=13, fontweight='bold', pad=20)
ax.legend(loc='upper right', bbox_to_anchor=(1.45, 1.15), fontsize=10)
ax.grid(True, alpha=0.4)

plt.tight_layout()
plt.savefig('model_radar.png', dpi=120, bbox_inches='tight')
plt.show()

print("Radar chart okuma:")
print("  Dis = Iyi, Ic = Kotu")
print("  Genis alan = genel olarak daha iyi model")
print("  Her eksen farkli bir metrik -> dengeli performans onemlidir")

```



Radar chart okuma:

Dis = İyi, Ic = Kotu

Genis alan = genel olarak daha iyi model

Her eksen farkli bir metrik -> dengeli performans onemlidir

BOLUM 5: HIPERPARAMETRE OPTIMIZASYONU

5.1 GridSearchCV Nedir?

Bir modelin hiperparametrelerini "deneme yanilma" ile secmek hem zaman alir hem de overfitting'e yol acar.

GridSearchCV su isi yapar:

1. Belirtilen parametre izgarasindaki **tum kombinasyonlari** dener
2. Her kombinasyon icin **k-fold cross-validation** uygular
3. En yuksek CV skorunu veren parametreleri otomatik secer

Cross-validation neden gerekli?

Test setini parametre seciminde kullanmak test setini "gorulmus" yapar.

CV egitim verisi icinde bir "ic test" mekanizmasi olusturur, test seti bagimsiz kalir.

5.2 Random Forest Onemli Hiperparametreleri

Parametre	Etkisi
<code>n_estimators</code>	Agac sayisi. Artinca daha kararli, daha yavas.
<code>max_depth</code>	Her agacin maksimum derinligi. Az = underfitting, Cok = overfitting.
<code>min_samples_split</code>	Bolmek icin gereken min ornek. Buyuk = daha basit agac.
<code>max_features</code>	Her bolmede denenen oznitelik sayisi. <code>sqrt</code> iyi baslangic.

```
In [19]: # — GridSearchCV ile Random Forest Optimizasyonu —————
# DIKKAT: n_jobs=1 kullaniyoruz.
# GridSearchCV(n_jobs=-1) + RF(n_jobs=-1) = nested parallelism = hata!
# Guvenli: ya GridSearchCV ya da RF'de paralellizm aktif olmalı.

parametre_izgarasi = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5],
    'max_features': ['sqrt', 'log2'],
}
# Toplam: 3 x 3 x 2 x 2 = 36 kombinasyon
# cv=5 fold: 36 x 5 = 180 model egitilecek

print("GridSearchCV baslatiyor... (36 kombinasyon x 5 fold = 180 fit)")
print("scoring='f1': Kanser tespitinde F1 daha anlamlı\n")

grid_rf = GridSearchCV(
    estimator = RandomForestClassifier(random_state=42),
    param_grid = parametre_izgarasi,
    cv = 5,
    scoring = 'f1',
    verbose = 1,
    n_jobs = 1
)
grid_rf.fit(Xk_tr_s, yk_tr)

print(f"\nEn iyi parametreler : {grid_rf.best_params_}")
print(f"En iyi CV F1 skoru : {grid_rf.best_score_:.4f}")

y_opt = grid_rf.predict(Xk_te_s)
print(f"\nTest sonuclari (optimize edilmiş model):")
print(f" Accuracy : {accuracy_score(yk_te, y_opt):.4f}")
print(f" Precision : {precision_score(yk_te, y_opt):.4f}")
print(f" Recall : {recall_score(yk_te, y_opt):.4f}")
print(f" F1-Score : {f1_score(yk_te, y_opt):.4f}")
```

GridSearchCV baslatiyor... (36 kombinasyon x 5 fold = 180 fit)
scoring='f1': Kanser tespitinde F1 daha anlamlı

Fitting 5 folds for each of 36 candidates, totalling 180 fits

En iyi parametreler : {'max_depth': None, 'max_features': 'sqrt', 'min_samples_split': 2, 'n_estimators': 200}
En iyi CV F1 skoru : 0.9684

Test sonuclari (optimize edilmiş model):

Accuracy : 0.9561
Precision : 0.9589
Recall : 0.9722
F1-Score : 0.9655

```
In [20]: # — GridSearch Sonuclarini Isi Haritasiyla Gorsellestir —
# Hangi parametre kombinasyonu en iyi skoru verdi?

import pandas as pd

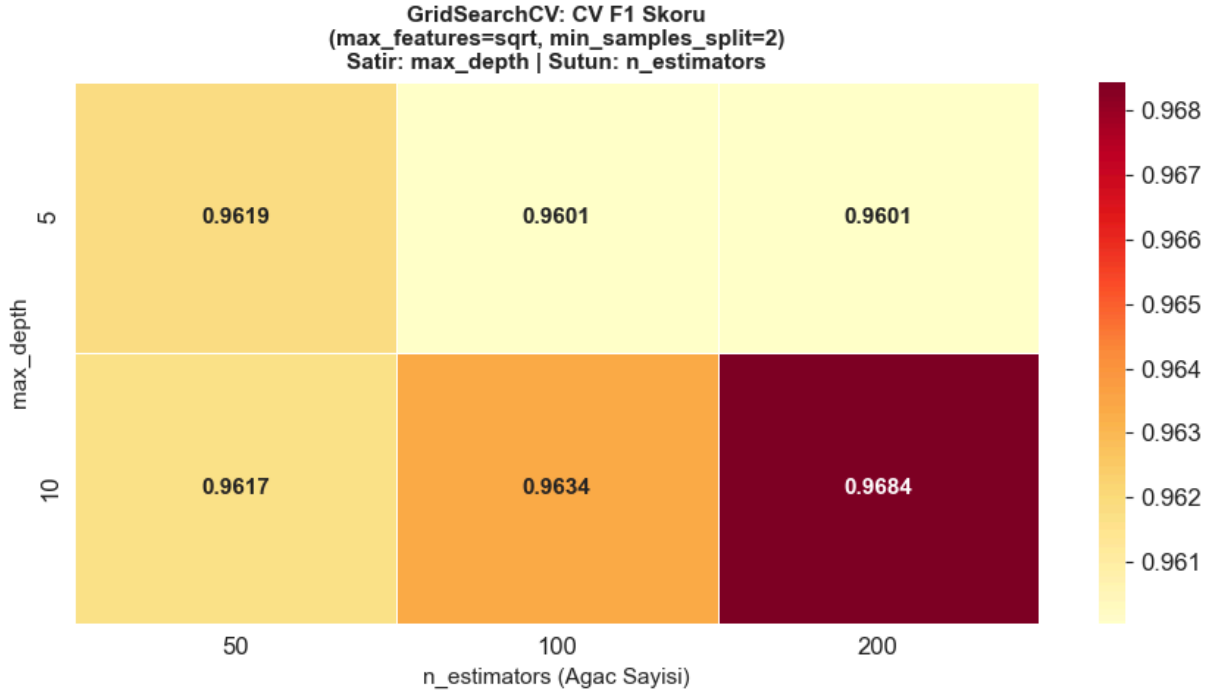
sonuclar_df = pd.DataFrame(grid_rf.cv_results_)

# n_estimators x max_depth icin pivot tablo (max_features='sqrt', min_samples_split
filtre = (
    (sonuclar_df['param_max_features'] == 'sqrt') &
    (sonuclar_df['param_min_samples_split'] == 2)
)
pivot = sonuclar_df[filtre].pivot_table(
    values='mean_test_score',
    index='param_max_depth',
    columns='param_n_estimators'
)

fig, ax = plt.subplots(figsize=(9, 5))
sns.heatmap(
    pivot, annot=True, fmt='.4f', cmap='YlOrRd', ax=ax,
    linewidths=0.5, annot_kws={'size': 11, 'weight': 'bold'}
)
ax.set_title(
    'GridSearchCV: CV F1 Skoru\n'
    '(max_features=sqrt, min_samples_split=2)\n'
    'Satir: max_depth | Sutun: n_estimators',
    fontsize=11, fontweight='bold'
)
ax.set_xlabel('n_estimators (Agac Sayisi)', fontsize=11)
ax.set_ylabel('max_depth', fontsize=11)

plt.tight_layout()
plt.savefig('gridsearch_heatmap.png', dpi=120, bbox_inches='tight')
plt.show()

print("Isi haritasi okuma:")
print(" Koyu renk = Yuksek F1 = Iyi kombinasyon")
print(" Bu grafik parametre secimini transparan sekilde gosterir.")
```



Isi haritasi okuma:

Koyu renk = Yuksek F1 = Iyi kombinasyon

Bu grafik parametre secimini transparan sekilde gosterir.

DERS 8: TAM OZET

Naive Bayes

Bayes teoremini kullanarak her sinibin olasiligini hesaplar, en yuksek olasikli sinifi secer.

Kilit kavramlar:

- **Bag of Words** — Kelimeleri sayar, sira onemssiz
- **Laplace duzeltmesi** — Sifir olasiligi onler (count + 1)
- $\text{Prior} \times \text{Likelihood} = \text{Posterior}$

Ne zaman kullan?

- Metin siniflandirma (spam, duygu analizi, haber kategorisi)
- Az veriyle hizli baseline model gerektiginde

Zayif yonleri:

- Kelime bagimsizlik varsayimi gercek dilde cogu zaman yanlis
- Sarkazm, negasyon, jargon yakalanamaz

Karar Agaclari

Ozyinelemeli Gini/Entropi minimizasyonula "evet/hayir" bolmeleri yapar.

Kilit kavramlar:

- Gini = 0 tam saf dugum (iyi)
- max_depth overfitting kontrolu saglar
- Budama (pruning) ile overfitting onlenir

Avantaj: Yorumlanabilir — kurallar okunabilir

Dezavantaj: Tek agac yuksek varyansa (overfitting'e) cok yatkın

Random Forest

Cok sayida karar agacini Bagging + rastgele oznitelik secimi ile egitir.

Kilit kavramlar:

- Bootstrap ornekleme — Her agac farkli veri goruyor
- Rastgele oznitelik — Agaclar korelasyonsuz, hatalar iptal oluyor
- Feature Importance — Hangi oznitelik daha onemli?

Ne zaman kullan?

- Genel amaclı siniflandirma ve regresyon
- Buyuk, gurultulu veri setlerinde
- Ilk deneme modeli olarak

Degerlendirme Metrikleri

Metrik	Formul	Ne Zaman?
Accuracy	$(TP+TN)/\text{Toplam}$	Dengeli veri
Precision	$TP/(TP+FP)$	FP maliyetliyse (spam filtresi)
Recall	$TP/(TP+FN)$	FN maliyetliyse (kanser tespiti)
F1-Score	$2PR/(P+R)$	Dengesiz veri, ikisi de onemli

Makale Baglantisi (Yahya et al., 2025 — ACL Workshop):

Arapca haber metinlerinde bias tespiti: RandomForest %93, XGBoost %92, GaussianNB %68
Buyuk dil modelleri (AraBERT %58, T5 %15) geleneksel ML'in gerisinde kaldi.

DERS: Her problem icin en karmasik model, en iyi model degildir!

-
- Not: Şebnem Paluluoğlu — Veri Analizi Okulu, Boğaziçi Üniversitesi*
Akademik referans: Yahya M. et al. (2025). Bias Detection in Media. ACL Workshop on Nakba Narratives.

-Memreu